confident-ai / **deepeval** Public

Code | Issues 210 | Pull requests 55 | Discussions | Actions

main | 293 Branches | 294 Tags | Go to file | Go to file | Code

penguine-ip updated guides ✕ 265bfbd · yesterday

.cursor-plugin	vibe koding	3 days ago
.github	add custom controller	4 days ago
.scripts/changelog	updated docs	4 days ago
.vscode	improve docs	4 months ago
assets	.	last month
deepeval	new release	2 days ago
demo_trace_scope	added tags and metadata to g-eval	2 days ago
docs	updated guides	yesterday
examples	added tags and metadata to g-eval	2 days ago
scripts	bumped default model to gpt-5.4	4 days ago
skills	reformat	3 days ago
tests	added tags and metadata to g-eval	2 days ago
.env.example	feat: add tests for openrouter and updat...	3 months ago
.gitignore	fix(gitignore): anchor Python lib/ rule to ...	last week
.pre-commit-config.yaml	fix: stage the pre commit config	7 months ago
CITATION.cff	new release	2 days ago
CONTRIBUTING.md	chore(docs): Update stalness policy wit...	5 months ago
LICENSE.md	Update LICENSE.md	2 years ago
MAINTAINERS.md	chore(docs): Update stalness policy wit...	5 months ago
MANIFEST.in	Inlcude txt files	2 years ago
README.md	added tags and metadata to g-eval	2 days ago
poetry.lock	chore: bump posthog to 7.x for Python >...	3 weeks ago
pyproject.toml	new release	2 days ago
simulate_conversations_example....	added tags and metadata to g-eval	2 days ago

README | Contributing | Apache-2.0 license

DeepEval Logo

The LLM Evaluation Framework

GITHUB TRENDING
#1 Repository Of The Day

discord-invite

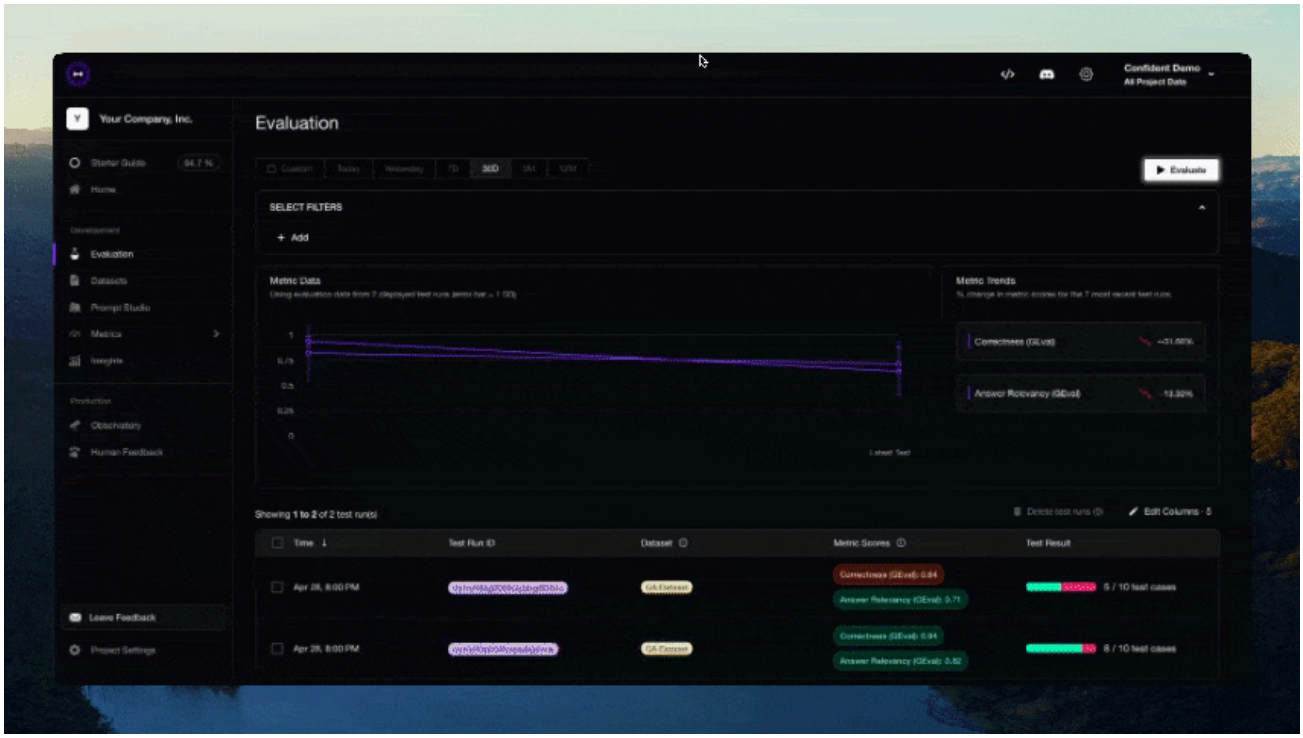
Documentation | **Metrics and Features** | **Getting Started** | **Integrations** | **Confident AI**

release v3.9.9 | Open in Colab | license Apache-2.0 | Follow @deepeval

Deutsch | Español | français | _____ | 한국어 | Português | Русский | ____

DeepEval is a simple-to-use, open-source LLM evaluation framework, for evaluating large-language model systems. It is similar to Pytest but specialized for unit testing LLM apps. DeepEval incorporates the latest research to run evals via metrics such as G-Eval, task completion, answer relevancy, hallucination, etc., which uses LLM-as-a-judge and other NLP models that run **locally on your machine**.

Whether you're building AI agents, RAG pipelines, or chatbots, implemented via LangChain or OpenAI, DeepEval has you covered. With it, you can easily determine the optimal models, prompts, and architecture to improve your AI quality, prevent prompt drifting, or even transition from OpenAI to Claude with confidence.

> Important
>
> Need a place for your DeepEval testing data to live 🏡 ❤️ ? Sign up to the DeepEval platform to compare iterations of your LLM app, generate & share testing reports, and more.
>
> 
>

> Want to talk LLM evaluation, need help picking metrics, or just to say hi? Come join our discord.

🔥 Metrics and Features

- 📐 Large variety of ready-to-use LLM eval metrics (all with explanations) powered by **ANY** LLM of your choice, statistical methods, or NLP models that run **locally on your machine** covering all use cases:
  - **Custom, All-Purpose Metrics:**
    - G-Eval — a research-backed LLM-as-a-judge metric for evaluating on any custom criteria with human-like accuracy
    - DAG — DeepEval's graph-based deterministic LLM-as-a-judge metric builder
  - ▶ **Agentic Metrics**
  - ▶ **RAG Metrics**
  - ▶ **Multi-Turn Metrics**
  - ▶ **MCP Metrics**
  - ▶ **Multimodal Metrics**
  - ▶ **Other Metrics**
- 🎯 Supports both end-to-end and component-level LLM evaluation.
- 🧩 Build your own custom metrics that are automatically integrated with DeepEval's ecosystem.
- 🔮 Generate both single and multi-turn synthetic datasets for evaluation.
- 🔗 Integrates seamlessly with **ANY** CI/CD environment.
- 🧬 Optimize prompts automatically based on evaluation results.
- 🏆 Easily benchmark **ANY** LLM on popular LLM benchmarks in under 10 lines of code., including MMLU, HellaSwag, DROP, BIG-Bench Hard, TruthfulQA, HumanEval, GSM8K.

🔌 Integrations

DeepEval plugs into any LLM framework — OpenAI Agents, LangChain, CrewAI, and more. To scale evals across your team — or let anyone run them without writing code — **Confident AI** gives you a native platform integration.

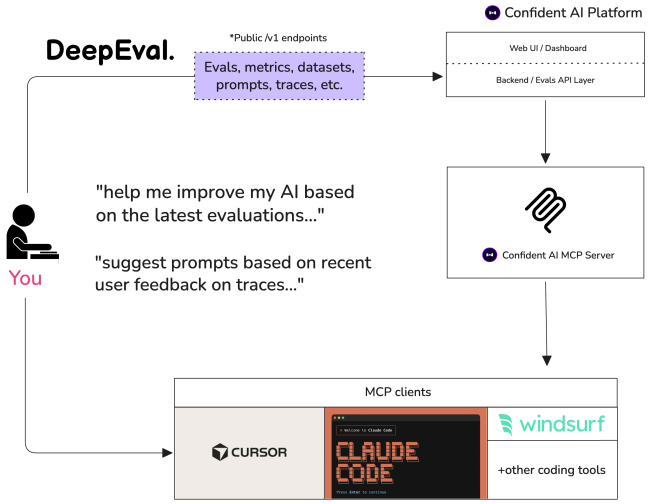
Frameworks

- OpenAI — evaluate and trace OpenAI applications via a client wrapper
- OpenAI Agents — evaluate OpenAI Agents end-to-end in under a minute
- LangChain — evaluate LangChain applications with a callback handler
- LangGraph — evaluate LangGraph agents with a callback handler
- Pydantic AI — evaluate Pydantic AI agents with type-safe validation
- CrewAI — evaluate CrewAI multi-agent systems
- Anthropic — evaluate and trace Claude applications via a client wrapper
- AWS AgentCore — evaluate agents deployed on Amazon AgentCore
- LlamaIndex — evaluate RAG applications built with LlamaIndex

☁️ Platform + Ecosystem

Confident AI is an all-in-one platform that integrates natively with DeepEval.

- Manage datasets, trace LLM applications, run evaluations, and monitor responses in production — all from one platform.
- Don't need a UI? Confident AI can also be your data persistant layer - run evals, pull datasets, and inspect traces straight from claude code, cursor, via Confident AI's MCP server.



🚀 QuickStart

Let's pretend your LLM application is a RAG based customer support chatbot; here's how DeepEval can help test what you've built.

Installation

Deepeval works with **Python>=3.9+**.

```
pip install -U deepeval
```

Create an account (highly recommended)

Using the `deepeval` platform will allow you to generate sharable testing reports on the cloud. It is free, takes no additional code to setup, and we highly recommend giving it a try.

To login, run:

```
deepeval login
```

Follow the instructions in the CLI to create an account, copy your API key, and paste it into the CLI. All test cases will automatically be logged (find more information on data privacy here).

Write your first test case

Create a test file:

```
touch test_chatbot.py
```

Open `test_chatbot.py` and write your first test case to run an **end-to-end** evaluation using DeepEval, which treats your LLM app as a black-box:

```
import pytest
from deepeval import assert_test
from deepeval.metrics import GEval
from deepeval.test_case import LLMTestCase, SingleTurnParams

def test_case():
    correctness_metric = GEval(
        name="Correctness",
        criteria="Determine if the 'actual output' is correct based on the 'expected output'.",
        evaluation_params=[SingleTurnParams.ACTUAL_OUTPUT, SingleTurnParams.EXPECTED_OUTPUT],
        threshold=0.5
    )
    test_case = LLMTestCase(
        input="What if these shoes don't fit?",
        # Replace this with the actual output from your LLM application
        actual_output="You have 30 days to get a full refund at no extra cost.",
        expected_output="We offer a 30-day full refund at no extra costs.",
        retrieval_context=["All customers are eligible for a 30 day full refund at no extra costs."]
    )
    assert_test(test_case, [correctness_metric])
```

Set your `OPENAI_API_KEY` as an environment variable (you can also evaluate using your own custom model, for more details visit [this part of our docs]):

```
export OPENAI_API_KEY="..."
```

And finally, run `test_chatbot.py` in the CLI:

```
deepeval test run test_chatbot.py
```

Congratulations! Your test case should have passed ✅ Let's breakdown what happened.

- The variable `input` mimics a user input, and `actual_output` is a placeholder for what your application's supposed to output based on this input.
- The variable `expected_output` represents the ideal answer for a given `input`, and `GEval` is a research-backed metric provided by `deepeval` for you to evaluate your LLM output's on any custom with human-like accuracy.
- In this example, the metric `criteria` is correctness of the `actual_output` based on the provided `expected_output`.
- All metric scores range from 0 - 1, which the `threshold=0.5` threshold ultimately determines if your test have passed or not.

[Read our documentation] for more information!

Evaluating Nested Components

Use the `@observe` decorator to trace components (LLM calls, retrievers, tool calls, agents) and apply metrics at the component level — no need to rewrite your codebase:

```
from deepeval.tracing import observe, update_current_span
from deepeval.test_case import LLMTestCase, SingleTurnParams
from deepeval.dataset import EvaluationDataset, Golden
from deepeval.metrics import GEval

correctness = GEval(
    name="Correctness",
    criteria="Determine if the 'actual output' is correct based on the 'expected output'.",
    evaluation_params=[SingleTurnParams.ACTUAL_OUTPUT, SingleTurnParams.EXPECTED_OUTPUT],
)

@observe(metrics=[correctness])
def inner_component():
    update_current_span(test_case=LLMTestCase(input="...", actual_output="..."))
    return "result"

@observe()
def llm_app(input: str):
    return inner_component()

dataset = EvaluationDataset(goldens=[Golden(input="Hi!")])
for golden in dataset.evals_iterator():
    llm_app(golden.input)
```

Learn more about component-level evaluations here.

Evaluate Without Pytest Integration

Alternatively, you can evaluate without Pytest, which is more suited for a notebook environment.

```
from deepeval import evaluate
from deepeval.metrics import AnswerRelevancyMetric
from deepeval.test_case import LLMTestCase

answer_relevancy_metric = AnswerRelevancyMetric(threshold=0.7)
test_case = LLMTestCase(
    input="What if these shoes don't fit?",
    # Replace this with the actual output from your LLM application
    actual_output="We offer a 30-day full refund at no extra costs.",
    retrieval_context=["All customers are eligible for a 30 day full refund at no extra costs."]
)
evaluate([test_case], [answer_relevancy_metric])
```

Using Standalone Metrics

DeepEval is extremely modular, making it easy for anyone to use any of our metrics. Continuing from the previous example:

```
from deepeval.metrics import AnswerRelevancyMetric
from deepeval.test_case import LLMTestCase

answer_relevancy_metric = AnswerRelevancyMetric(threshold=0.7)
```

```
test_case = LLMTestCase(
    input="What if these shoes don't fit?",
    # Replace this with the actual output from your LLM application
    actual_output="We offer a 30-day full refund at no extra costs.",
    retrieval_context=["All customers are eligible for a 30 day full refund at no extra costs."]
)

answer_relevancy_metric.measure(test_case)
print(answer_relevancy_metric.score)
# All metrics also offer an explanation
print(answer_relevancy_metric.reason)
```

Note that some metrics are for RAG pipelines, while others are for fine-tuning. Make sure to use our docs to pick the right one for your use case.

Evaluating a Dataset / Test Cases in Bulk

In DeepEval, a dataset is simply a collection of test cases. Here is how you can evaluate these in bulk:

```
import pytest
from deepeval import assert_test
from deepeval.dataset import EvaluationDataset, Golden
from deepeval.metrics import AnswerRelevancyMetric
from deepeval.test_case import LLMTestCase

dataset = EvaluationDataset(goldens=[Golden(input="What's the weather like today?")])

for golden in dataset.goldens:
    test_case = LLMTestCase(
        input=golden.input,
        actual_output=your_llm_app(golden.input)
    )
    dataset.add_test_case(test_case)

@pytest.mark.parametrize(
    "test_case",
    dataset.test_cases,
)
def test_customer_chatbot(test_case: LLMTestCase):
    answer_relevancy_metric = AnswerRelevancyMetric(threshold=0.5)
    assert_test(test_case, [answer_relevancy_metric])
```

```
# Run this in the CLI, you can also add an optional -n flag to run tests in parallel
deepeval test run test_<filename>.py -n 4
```

Alternatively, although we recommend using `deepeval test run`, you can evaluate a dataset/test cases without using our Pytest integration:

```
from deepeval import evaluate
...

evaluate(dataset, [answer_relevancy_metric])
```

A Note on Env Variables (.env / .env.local)

DeepEval auto-loads `.env.local` then `.env` from the current working directory **at import time**. **Precedence:** process env -> `.env.local` -> `.env`. Opt out with `DEEPEVAL_DISABLE_DOTENV=1`.

```
cp .env.example .env.local
# then edit .env.local (ignored by git)
```

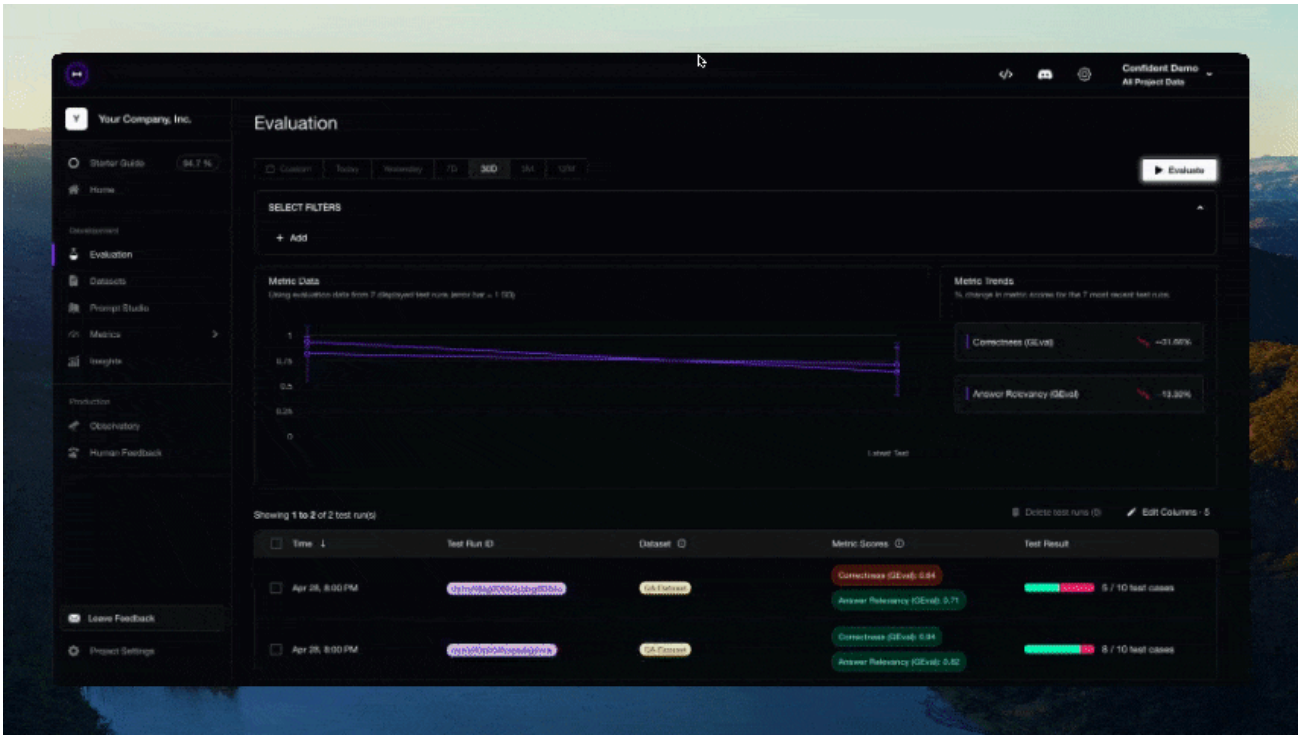
DeepEval With Confident AI

Confident AI is an all-in-one platform to manage datasets, trace LLM applications, and run evaluations in production. Log in from the CLI to get started:

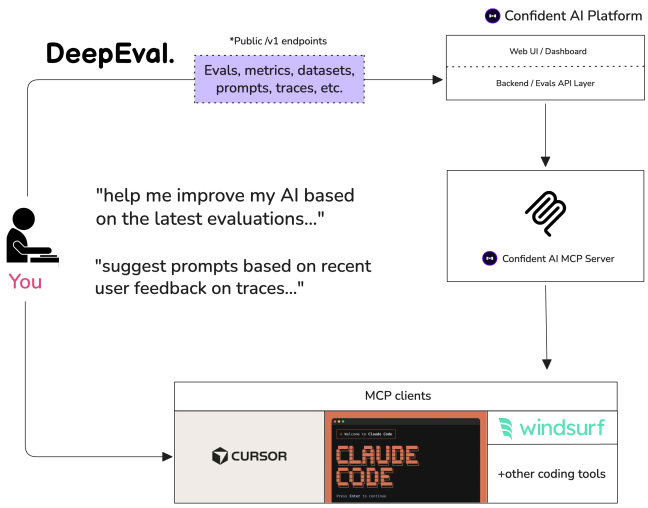
```
deepeval login
```

Then run your tests as usual — results are automatically synced to the platform:

```
deepeval test run test_chatbot.py
```



Prefer to stay in your IDE? Use DeepEval via Confident AI's MCP server as the persistent layer to run evals, pull datasets, and inspect traces without leaving your editor.



Everything on Confident AI is available here.

Contributing

Please read CONTRIBUTING.md for details on our code of conduct, and the process for submitting pull requests to us.

Roadmap

Features:

- [x] Integration with Confident AI
- [x] Implement G-Eval
- [x] Implement RAG metrics
- [x] Implement Conversational metrics
- [x] Evaluation Dataset Creation
- [x] Red-Teaming

Releases 53

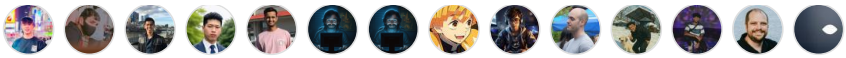
🎉 **Metrics for AI agents, multi-turn synthetic data generation, and more!** Latest
on Dec 1, 2025

+ 52 releases

Packages

No packages published

Contributors 256



+ 242 contributors

Languages

- **Python** 100.0%